lab3

March 10, 2026

1 Głębokie Uczenie w Praktyce

1.1 Laboratorium 2

1.1.1 Mateusz Horczak

[16]:
```
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Embedding, Input
from tensorflow.keras.datasets import imdb
from tensorflow.keras import preprocessing
from scipy.spatial import distance
```

Ćwiczenie 1 Word Embedding - osadzenie słów

[17]:
```
docs = ['Dobrze zrobione!', 'Dobra robota', 'Znakomity wysiłek', 'fajna␣
 ↪robota', 'Wspaniałe!',
       'Słabe', 'mały wysiłek!', 'nie dobra', 'słaba praca', 'Można było␣
 ↪lepiej zrobić.']
labels = np.array([1, 1, 1, 1, 1, 0, 0, 0, 0, 0])
```

[18]:
```
t = Tokenizer()
t.fit_on_texts(docs)
vocab_size = len(t.word_index) + 1
print("vocab_size:", vocab_size)
```

```
vocab_size: 18
```

[19]:
```
encoded_docs = t.texts_to_sequences(docs)
print("encoded_docs:", encoded_docs)
```

```
encoded_docs: [[4, 5], [1, 2], [6, 3], [7, 2], [8], [9], [10, 3], [11, 1], [12,
13], [14, 15, 16, 17]]
```

[20]:
```
max_length = 4
padded_docs = pad_sequences(encoded_docs, maxlen=max_length, padding='post')
print("padded_docs:\n", padded_docs)
```

```
padded_docs:
 [[ 4  5  0  0]
 [ 1  2  0  0]
 [ 6  3  0  0]
 [ 7  2  0  0]
 [ 8  0  0  0]
 [ 9  0  0  0]
 [10  3  0  0]
 [11  1  0  0]
 [12 13  0  0]
 [14 15 16 17]]
```

[21]:
```
model = Sequential()
model.add(Input(shape=(max_length,)))
model.add(Embedding(vocab_size, 8, name='emb1'))
model.add(Flatten())
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam', loss='binary_crossentropy',␣
 ↪metrics=['accuracy'])
print(model.summary())
```

Model: "sequential_13"

Layer (type)	Output Shape	Param #
emb1 (Embedding)	(None, 4, 8)	144
flatten_9 (Flatten)	(None, 32)	0
dense_13 (Dense)	(None, 1)	33

Total params: 177 (708.00 B)

Trainable params: 177 (708.00 B)

Non-trainable params: 0 (0.00 B)

None

[22]:
```
model.fit(padded_docs, labels, epochs=50, verbose=1)
```

```
Epoch 1/50
1/1             1s 699ms/step -
```

```
accuracy: 0.4000 - loss: 0.6976
Epoch 2/50
1/1             0s 47ms/step -
accuracy: 0.4000 - loss: 0.6960
Epoch 3/50
1/1             0s 56ms/step -
accuracy: 0.4000 - loss: 0.6943
Epoch 4/50
1/1             0s 47ms/step -
accuracy: 0.4000 - loss: 0.6927
Epoch 5/50
1/1             0s 49ms/step -
accuracy: 0.4000 - loss: 0.6911
Epoch 6/50
1/1             0s 44ms/step -
accuracy: 0.5000 - loss: 0.6895
Epoch 7/50
1/1             0s 60ms/step -
accuracy: 0.5000 - loss: 0.6879
Epoch 8/50
1/1             0s 57ms/step -
accuracy: 0.6000 - loss: 0.6863
Epoch 9/50
1/1             0s 53ms/step -
accuracy: 0.7000 - loss: 0.6847
Epoch 10/50
1/1             0s 46ms/step -
accuracy: 0.7000 - loss: 0.6831
Epoch 11/50
1/1             0s 46ms/step -
accuracy: 0.8000 - loss: 0.6815
Epoch 12/50
1/1             0s 48ms/step -
accuracy: 0.9000 - loss: 0.6798
Epoch 13/50
1/1             0s 45ms/step -
accuracy: 0.9000 - loss: 0.6782
Epoch 14/50
1/1             0s 56ms/step -
accuracy: 1.0000 - loss: 0.6766
Epoch 15/50
1/1             0s 56ms/step -
accuracy: 1.0000 - loss: 0.6750
Epoch 16/50
1/1             0s 54ms/step -
accuracy: 1.0000 - loss: 0.6733
Epoch 17/50
1/1             0s 57ms/step -
```

```
accuracy: 1.0000 - loss: 0.6717
Epoch 18/50
1/1             0s 53ms/step -
accuracy: 1.0000 - loss: 0.6701
Epoch 19/50
1/1             0s 49ms/step -
accuracy: 1.0000 - loss: 0.6684
Epoch 20/50
1/1             0s 53ms/step -
accuracy: 1.0000 - loss: 0.6667
Epoch 21/50
1/1             0s 49ms/step -
accuracy: 1.0000 - loss: 0.6651
Epoch 22/50
1/1             0s 44ms/step -
accuracy: 1.0000 - loss: 0.6634
Epoch 23/50
1/1             0s 43ms/step -
accuracy: 1.0000 - loss: 0.6617
Epoch 24/50
1/1             0s 44ms/step -
accuracy: 1.0000 - loss: 0.6600
Epoch 25/50
1/1             0s 43ms/step -
accuracy: 1.0000 - loss: 0.6583
Epoch 26/50
1/1             0s 50ms/step -
accuracy: 1.0000 - loss: 0.6565
Epoch 27/50
1/1             0s 62ms/step -
accuracy: 1.0000 - loss: 0.6548
Epoch 28/50
1/1             0s 49ms/step -
accuracy: 1.0000 - loss: 0.6530
Epoch 29/50
1/1             0s 48ms/step -
accuracy: 1.0000 - loss: 0.6512
Epoch 30/50
1/1             0s 43ms/step -
accuracy: 1.0000 - loss: 0.6495
Epoch 31/50
1/1             0s 63ms/step -
accuracy: 1.0000 - loss: 0.6477
Epoch 32/50
1/1             0s 60ms/step -
accuracy: 1.0000 - loss: 0.6458
Epoch 33/50
1/1             0s 42ms/step -
```

```
accuracy: 1.0000 - loss: 0.6440
Epoch 34/50
1/1             0s 52ms/step -
accuracy: 1.0000 - loss: 0.6422
Epoch 35/50
1/1             0s 52ms/step -
accuracy: 1.0000 - loss: 0.6403
Epoch 36/50
1/1             0s 46ms/step -
accuracy: 1.0000 - loss: 0.6384
Epoch 37/50
1/1             0s 53ms/step -
accuracy: 1.0000 - loss: 0.6365
Epoch 38/50
1/1             0s 55ms/step -
accuracy: 1.0000 - loss: 0.6346
Epoch 39/50
1/1             0s 47ms/step -
accuracy: 1.0000 - loss: 0.6327
Epoch 40/50
1/1             0s 49ms/step -
accuracy: 1.0000 - loss: 0.6307
Epoch 41/50
1/1             0s 45ms/step -
accuracy: 1.0000 - loss: 0.6288
Epoch 42/50
1/1             0s 46ms/step -
accuracy: 1.0000 - loss: 0.6268
Epoch 43/50
1/1             0s 48ms/step -
accuracy: 1.0000 - loss: 0.6248
Epoch 44/50
1/1             0s 53ms/step -
accuracy: 1.0000 - loss: 0.6228
Epoch 45/50
1/1             0s 52ms/step -
accuracy: 1.0000 - loss: 0.6208
Epoch 46/50
1/1             0s 53ms/step -
accuracy: 1.0000 - loss: 0.6187
Epoch 47/50
1/1             0s 49ms/step -
accuracy: 1.0000 - loss: 0.6167
Epoch 48/50
1/1             0s 46ms/step -
accuracy: 1.0000 - loss: 0.6146
Epoch 49/50
1/1             0s 43ms/step -
```

```
accuracy: 1.0000 - loss: 0.6125
Epoch 50/50
1/1             0s 45ms/step -
accuracy: 1.0000 - loss: 0.6104
```

[22]:
```
<keras.src.callbacks.history.History at 0x7f0bc3478e60>
```

Ćwiczenie 2 Sprawdzenie jak wyglądają osadzone wektory i policzenie odległości między nimi

[23]:
```
model2 = Sequential()
model2.add(Input(shape=(max_length,)))
model2.add(Embedding(vocab_size, 8))
model2.compile(optimizer='adam', loss='binary_crossentropy',␣
 ↪metrics=['accuracy'])
model2.summary()
```

```
Model: "sequential_14"
```

Layer (type)	Output Shape	Param #
embedding_9 (Embedding)	(None, 4, 8)	144

```
Total params: 144 (576.00 B)

Trainable params: 144 (576.00 B)

Non-trainable params: 0 (0.00 B)
```

[24]:
```
model2.layers[0].set_weights(model.layers[0].get_weights()) # kopiowanie wag z␣
 ↪wytrenowanej wartstwy
embeddings = model2.layers[0].get_weights()[0] # pobieranie rzeczywistych␣
 ↪wektorów

# Funkcja do obliczania odległości cosinusowej między wektorami słów
def distance_between_words(word, word_dict, embeddings):
    words = list(word_dict.values())
    if word not in words:
        print("Słowo nie istnieje w słowniku")
        return

    target_index = words.index(word)
```

```
    target_vector = embeddings[target_index + 1]  # Indeksy słów zaczynają się␣
↪od 1

    results = []
    for i, w in enumerate(words):
        vec = embeddings[i + 1]
        dist = distance.cosine(target_vector, vec)
        results.append((w, dist))

    results.sort(key=lambda x: x[1])
    print("****************************")
    print(f"Najbliższe wektory do słowa: {word}")
    print("****************************\n")
    for w, d in results:
        print(f"{w:12s} {d:.3f}")
```

[25]:
```
word_dict = t.word_index
```

[26]:
```
distance_between_words("fajna", {v: k for k, v in word_dict.items()},␣
↪embeddings)
```

```
****************************
Najbliższe wektory do słowa: fajna
****************************

fajna        0.000
wspaniałe    0.209
dobrze       0.212
znakomity    0.268
dobra        0.764
robota       0.813
zrobić       0.819
zrobione     0.886
wysiłek      0.917
praca        0.984
było         1.028
lepiej       1.322
można        1.683
mały         1.748
nie          1.819
słaba        1.850
słabe        1.889
```

[27]:
```
distance_between_words("dobra", {v: k for k, v in word_dict.items()},␣
↪embeddings)
```

```
****************************
Najbliższe wektory do słowa: dobra
```

```
**************************

dobra        0.000
było         0.272
dobrze       0.546
zrobić       0.698
praca        0.733
fajna        0.764
znakomity    0.835
wysiłek      0.921
słaba        1.006
mały         1.112
wspaniałe    1.196
słabe        1.331
lepiej       1.346
nie          1.356
można        1.375
zrobione     1.439
robota       1.627
```

[28]:
```
distance_between_words("słaba", {v: k for k, v in word_dict.items()},␣
↪embeddings)
```

```
**************************
Najbliższe wektory do słowa: słaba
**************************

słaba        0.000
mały         0.096
słabe        0.180
nie          0.281
można        0.386
lepiej       0.515
było         0.643
wysiłek      0.723
praca        0.887
dobra        1.006
zrobione     1.126
zrobić       1.221
robota       1.300
znakomity    1.730
dobrze       1.848
fajna        1.850
wspaniałe    1.891
```

Ćwiczenie 3 Uczenie osadzeń słów i trenowanie warstwy Dense z użyciem zbioru IMDB

[29]:
```
max_features = 10000

(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_features)
print("x_train shape:", x_train.shape)
print("x_test shape:", x_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
x_train shape: (25000,)
x_test shape: (25000,)
y_train shape: (25000,)
y_test shape: (25000,)
```

[30]:
```
# Funkcja do trenowania modelu z embeddingami
def train_embedding_model(maxlen, output_dim):
    x_train_padded = pad_sequences(x_train, maxlen=maxlen)
    x_test_padded = pad_sequences(x_test, maxlen=maxlen)

    model = Sequential()
    model.add(Input(shape=(maxlen,)))
    model.add(Embedding(max_features, output_dim))
    model.add(Flatten())
    model.add(Dense(1, activation='sigmoid'))

    model.compile(optimizer='rmsprop', loss='binary_crossentropy',␣
 ↪metrics=['accuracy'])
    print(f"\nModel summary for maxlen={maxlen}, output_dim={output_dim}:")
    model.summary()

    history = model.fit(x_train_padded, y_train, epochs=10, batch_size=32,␣
 ↪validation_split=0.2, verbose=1)
    test_loss, test_acc = model.evaluate(x_test_padded, y_test)
    print(f"Test accuracy: {test_acc:.4f}")
    return history, test_acc

# Funkcja do trenowania modelu z one-hot
def train_one_hot_model(maxlen):
    tokenizer = Tokenizer(num_words=max_features)
    tokenizer.fit_on_texts([" ".join(map(str, seq)) for seq in x_train])
    x_train_onehot = tokenizer.texts_to_matrix([" ".join(map(str, seq[:
 ↪maxlen])) for seq in x_train], mode='binary')
    x_test_onehot = tokenizer.texts_to_matrix([" ".join(map(str, seq[:maxlen]))␣
 ↪for seq in x_test], mode='binary')

    model = Sequential()
    model.add(Input(shape=(max_features,)))
    model.add(Dense(128, activation='relu'))
```

```
    model.add(Dense(1, activation='sigmoid'))

    model.compile(optimizer='rmsprop', loss='binary_crossentropy',␣
 ↪metrics=['accuracy'])
    print(f"\nOne-hot model summary for maxlen={maxlen}:")
    model.summary()

    history = model.fit(x_train_onehot, y_train, epochs=10, batch_size=32,␣
 ↪validation_split=0.2, verbose=1)
    test_loss, test_acc = model.evaluate(x_test_onehot, y_test)
    print(f"One-hot test accuracy: {test_acc:.4f}")
    return history, test_acc
```

[31]:
```
print("Porównanie z one-hot:")
_, one_hot_acc = train_one_hot_model(20)
```

```
Porównanie z one-hot:

One-hot model summary for maxlen=20:
Model: "sequential_15"
```

Layer (type)	Output Shape	Param #
dense_14 (Dense)	(None, 128)	1,280,128
dense_15 (Dense)	(None, 1)	129

```
Total params: 1,280,257 (4.88 MB)

Trainable params: 1,280,257 (4.88 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10
625/625             10s 15ms/step -
accuracy: 0.6743 - loss: 0.5953 - val_accuracy: 0.7066 - val_loss: 0.5574
Epoch 2/10
625/625             9s 14ms/step -
accuracy: 0.7574 - loss: 0.4990 - val_accuracy: 0.7188 - val_loss: 0.5505
Epoch 3/10
625/625             8s 13ms/step -
accuracy: 0.7869 - loss: 0.4540 - val_accuracy: 0.7160 - val_loss: 0.5574
Epoch 4/10
```

```
625/625             8s 13ms/step -
accuracy: 0.8148 - loss: 0.4142 - val_accuracy: 0.7164 - val_loss: 0.5745
Epoch 5/10
625/625             8s 13ms/step -
accuracy: 0.8438 - loss: 0.3737 - val_accuracy: 0.7164 - val_loss: 0.5908
Epoch 6/10
625/625             8s 13ms/step -
accuracy: 0.8700 - loss: 0.3307 - val_accuracy: 0.7136 - val_loss: 0.6226
Epoch 7/10
625/625             8s 13ms/step -
accuracy: 0.8927 - loss: 0.2890 - val_accuracy: 0.7124 - val_loss: 0.6494
Epoch 8/10
625/625             8s 13ms/step -
accuracy: 0.9119 - loss: 0.2503 - val_accuracy: 0.7116 - val_loss: 0.6776
Epoch 9/10
625/625             9s 14ms/step -
accuracy: 0.9290 - loss: 0.2137 - val_accuracy: 0.7140 - val_loss: 0.7251
Epoch 10/10
625/625             8s 13ms/step -
accuracy: 0.9438 - loss: 0.1797 - val_accuracy: 0.6988 - val_loss: 0.7664
782/782             3s 3ms/step -
accuracy: 0.6922 - loss: 0.8059
One-hot test accuracy: 0.6922
```

[32]:
```
print("Embedding - różne maxlen (output_dim=16):")
for maxlen in [20, 50, 100, 200]:
    _, emb_acc = train_embedding_model(maxlen, 16)
    print(f"Embedding (maxlen={maxlen}) vs One-hot: {emb_acc:.4f} vs␣
 ↪{one_hot_acc:.4f}")
```

```
Embedding - różne maxlen (output_dim=16):

Model summary for maxlen=20, output_dim=16:
```

Model: "sequential_16"

Layer (type)	Output Shape	Param #
embedding_10 (Embedding)	(None, 20, 16)	160,000
flatten_10 (Flatten)	(None, 320)	0
dense_16 (Dense)	(None, 1)	321

Total params: 160,321 (626.25 KB)

```
 Trainable params: 160,321 (626.25 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/10
625/625             3s 4ms/step -
accuracy: 0.6589 - loss: 0.6447 - val_accuracy: 0.7106 - val_loss: 0.5718
Epoch 2/10
625/625             3s 4ms/step -
accuracy: 0.7639 - loss: 0.5002 - val_accuracy: 0.7416 - val_loss: 0.5078
Epoch 3/10
625/625             3s 4ms/step -
accuracy: 0.8012 - loss: 0.4343 - val_accuracy: 0.7528 - val_loss: 0.4953
Epoch 4/10
625/625             3s 5ms/step -
accuracy: 0.8241 - loss: 0.3956 - val_accuracy: 0.7544 - val_loss: 0.4984
Epoch 5/10
625/625             3s 4ms/step -
accuracy: 0.8423 - loss: 0.3644 - val_accuracy: 0.7548 - val_loss: 0.5035
Epoch 6/10
625/625             2s 4ms/step -
accuracy: 0.8577 - loss: 0.3349 - val_accuracy: 0.7490 - val_loss: 0.5145
Epoch 7/10
625/625             2s 4ms/step -
accuracy: 0.8748 - loss: 0.3076 - val_accuracy: 0.7482 - val_loss: 0.5209
Epoch 8/10
625/625             2s 4ms/step -
accuracy: 0.8885 - loss: 0.2809 - val_accuracy: 0.7464 - val_loss: 0.5348
Epoch 9/10
625/625             2s 4ms/step -
accuracy: 0.9043 - loss: 0.2549 - val_accuracy: 0.7448 - val_loss: 0.5477
Epoch 10/10
625/625             2s 4ms/step -
accuracy: 0.9136 - loss: 0.2308 - val_accuracy: 0.7410 - val_loss: 0.5624
782/782             1s 2ms/step -
accuracy: 0.7487 - loss: 0.5507
Test accuracy: 0.7487
Embedding (maxlen=20) vs One-hot: 0.7487 vs 0.6922

Model summary for maxlen=50, output_dim=16:

Model: "sequential_17"

 Layer (type)                    Output Shape                  Param #

 embedding_11 (Embedding)        (None, 50, 16)                160,000
```

```
 flatten_11 (Flatten)              (None, 800)                       0

 dense_17 (Dense)                  (None, 1)                       801


 Total params: 160,801 (628.13 KB)

 Trainable params: 160,801 (628.13 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/10
625/625             5s 7ms/step -
accuracy: 0.6716 - loss: 0.6216 - val_accuracy: 0.7688 - val_loss: 0.5002
Epoch 2/10
625/625             3s 5ms/step -
accuracy: 0.8142 - loss: 0.4185 - val_accuracy: 0.8024 - val_loss: 0.4172
Epoch 3/10
625/625             3s 5ms/step -
accuracy: 0.8501 - loss: 0.3470 - val_accuracy: 0.8108 - val_loss: 0.4062
Epoch 4/10
625/625             3s 5ms/step -
accuracy: 0.8730 - loss: 0.3064 - val_accuracy: 0.8110 - val_loss: 0.4045
Epoch 5/10
625/625             5s 5ms/step -
accuracy: 0.8906 - loss: 0.2720 - val_accuracy: 0.8086 - val_loss: 0.4136
Epoch 6/10
625/625             5s 5ms/step -
accuracy: 0.9081 - loss: 0.2394 - val_accuracy: 0.8102 - val_loss: 0.4160
Epoch 7/10
625/625             3s 4ms/step -
accuracy: 0.9244 - loss: 0.2069 - val_accuracy: 0.8048 - val_loss: 0.4327
Epoch 8/10
625/625             5s 5ms/step -
accuracy: 0.9396 - loss: 0.1766 - val_accuracy: 0.8034 - val_loss: 0.4436
Epoch 9/10
625/625             3s 5ms/step -
accuracy: 0.9530 - loss: 0.1477 - val_accuracy: 0.7994 - val_loss: 0.4609
Epoch 10/10
625/625             3s 4ms/step -
accuracy: 0.9642 - loss: 0.1214 - val_accuracy: 0.8006 - val_loss: 0.4790
782/782             2s 2ms/step -
accuracy: 0.8053 - loss: 0.4685
Test accuracy: 0.8053
Embedding (maxlen=50) vs One-hot: 0.8053 vs 0.6922
```

```
Model summary for maxlen=100, output_dim=16:
Model: "sequential_18"

 Layer (type)                    Output Shape                  Param #

 embedding_12 (Embedding)        (None, 100, 16)               160,000

 flatten_12 (Flatten)            (None, 1600)                        0

 dense_18 (Dense)                (None, 1)                       1,601


 Total params: 161,601 (631.25 KB)

 Trainable params: 161,601 (631.25 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/10
625/625             4s 5ms/step -
accuracy: 0.7088 - loss: 0.5825 - val_accuracy: 0.8130 - val_loss: 0.4226
Epoch 2/10
625/625             3s 5ms/step -
accuracy: 0.8558 - loss: 0.3452 - val_accuracy: 0.8430 - val_loss: 0.3463
Epoch 3/10
625/625             3s 5ms/step -
accuracy: 0.8877 - loss: 0.2740 - val_accuracy: 0.8498 - val_loss: 0.3337
Epoch 4/10
625/625             3s 5ms/step -
accuracy: 0.9105 - loss: 0.2301 - val_accuracy: 0.8496 - val_loss: 0.3454
Epoch 5/10
625/625             4s 6ms/step -
accuracy: 0.9290 - loss: 0.1934 - val_accuracy: 0.8530 - val_loss: 0.3419
Epoch 6/10
625/625             4s 6ms/step -
accuracy: 0.9449 - loss: 0.1612 - val_accuracy: 0.8482 - val_loss: 0.3552
Epoch 7/10
625/625             3s 5ms/step -
accuracy: 0.9574 - loss: 0.1310 - val_accuracy: 0.8476 - val_loss: 0.3689
Epoch 8/10
625/625             3s 5ms/step -
accuracy: 0.9687 - loss: 0.1033 - val_accuracy: 0.8430 - val_loss: 0.3848
Epoch 9/10
```

```
625/625             3s 5ms/step -
accuracy: 0.9793 - loss: 0.0796 - val_accuracy: 0.8408 - val_loss: 0.4022
Epoch 10/10
625/625             3s 5ms/step -
accuracy: 0.9862 - loss: 0.0598 - val_accuracy: 0.8408 - val_loss: 0.4210
782/782             2s 2ms/step -
accuracy: 0.8381 - loss: 0.4284
Test accuracy: 0.8381
Embedding (maxlen=100) vs One-hot: 0.8381 vs 0.6922

Model summary for maxlen=200, output_dim=16:
```

Model: "sequential_19"

Layer (type)	Output Shape	Param #
embedding_13 (Embedding)	(None, 200, 16)	160,000
flatten_13 (Flatten)	(None, 3200)	0
dense_19 (Dense)	(None, 1)	3,201

Total params: 163,201 (637.50 KB)

Trainable params: 163,201 (637.50 KB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
625/625             5s 6ms/step -
accuracy: 0.7135 - loss: 0.5594 - val_accuracy: 0.8342 - val_loss: 0.3886
Epoch 2/10
625/625             4s 7ms/step -
accuracy: 0.8688 - loss: 0.3135 - val_accuracy: 0.8566 - val_loss: 0.3347
Epoch 3/10
625/625             4s 6ms/step -
accuracy: 0.9026 - loss: 0.2458 - val_accuracy: 0.8738 - val_loss: 0.2977
Epoch 4/10
625/625             4s 6ms/step -
accuracy: 0.9240 - loss: 0.1992 - val_accuracy: 0.8600 - val_loss: 0.3364
Epoch 5/10
625/625             4s 6ms/step -
accuracy: 0.9413 - loss: 0.1606 - val_accuracy: 0.8704 - val_loss: 0.3146
Epoch 6/10
```

```
625/625             4s 6ms/step -
accuracy: 0.9579 - loss: 0.1277 - val_accuracy: 0.8670 - val_loss: 0.3236
Epoch 7/10
625/625             5s 8ms/step -
accuracy: 0.9704 - loss: 0.0975 - val_accuracy: 0.8600 - val_loss: 0.3620
Epoch 8/10
625/625             4s 6ms/step -
accuracy: 0.9801 - loss: 0.0734 - val_accuracy: 0.8680 - val_loss: 0.3443
Epoch 9/10
625/625             4s 6ms/step -
accuracy: 0.9869 - loss: 0.0536 - val_accuracy: 0.8636 - val_loss: 0.3843
Epoch 10/10
625/625             4s 6ms/step -
accuracy: 0.9919 - loss: 0.0386 - val_accuracy: 0.8622 - val_loss: 0.4048
782/782             2s 2ms/step -
accuracy: 0.8557 - loss: 0.4095
Test accuracy: 0.8557
Embedding (maxlen=200) vs One-hot: 0.8557 vs 0.6922
```

[33]:
```
print("Embedding - różne output_dim (maxlen=200):")
for output_dim in [8, 16, 64]:
    _, emb_acc = train_embedding_model(200, output_dim)
    print(f"Embedding (output_dim={output_dim}) vs One-hot: {emb_acc:.4f} vs␣
 ↪{one_hot_acc:.4f}")
```

```
Embedding - różne output_dim (maxlen=200):

Model summary for maxlen=200, output_dim=8:
```

Model: "sequential_20"

Layer (type)	Output Shape	Param #
embedding_14 (Embedding)	(None, 200, 8)	80,000
flatten_14 (Flatten)	(None, 1600)	0
dense_20 (Dense)	(None, 1)	1,601

Total params: 81,601 (318.75 KB)

Trainable params: 81,601 (318.75 KB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
625/625             4s 5ms/step -
accuracy: 0.6930 - loss: 0.5933 - val_accuracy: 0.8222 - val_loss: 0.4196
Epoch 2/10
625/625             3s 5ms/step -
accuracy: 0.8645 - loss: 0.3339 - val_accuracy: 0.8600 - val_loss: 0.3229
Epoch 3/10
625/625             5s 5ms/step -
accuracy: 0.8940 - loss: 0.2607 - val_accuracy: 0.8588 - val_loss: 0.3255
Epoch 4/10
625/625             3s 5ms/step -
accuracy: 0.9123 - loss: 0.2226 - val_accuracy: 0.8786 - val_loss: 0.2953
Epoch 5/10
625/625             2s 4ms/step -
accuracy: 0.9237 - loss: 0.1959 - val_accuracy: 0.8798 - val_loss: 0.2938
Epoch 6/10
625/625             2s 4ms/step -
accuracy: 0.9360 - loss: 0.1716 - val_accuracy: 0.8772 - val_loss: 0.2992
Epoch 7/10
625/625             2s 4ms/step -
accuracy: 0.9457 - loss: 0.1481 - val_accuracy: 0.8710 - val_loss: 0.3180
Epoch 8/10
625/625             2s 4ms/step -
accuracy: 0.9546 - loss: 0.1288 - val_accuracy: 0.8704 - val_loss: 0.3218
Epoch 9/10
625/625             2s 4ms/step -
accuracy: 0.9633 - loss: 0.1097 - val_accuracy: 0.8750 - val_loss: 0.3266
Epoch 10/10
625/625             2s 4ms/step -
accuracy: 0.9704 - loss: 0.0954 - val_accuracy: 0.8678 - val_loss: 0.3600
782/782             1s 2ms/step -
accuracy: 0.8662 - loss: 0.3526
Test accuracy: 0.8662
Embedding (output_dim=8) vs One-hot: 0.8662 vs 0.6922

Model summary for maxlen=200, output_dim=16:
```

Model: "sequential_21"

Layer (type)	Output Shape	Param #
embedding_15 (Embedding)	(None, 200, 16)	160,000
flatten_15 (Flatten)	(None, 3200)	0
dense_21 (Dense)	(None, 1)	3,201

```
 Total params: 163,201 (637.50 KB)

 Trainable params: 163,201 (637.50 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/10
625/625             4s 6ms/step -
accuracy: 0.7192 - loss: 0.5512 - val_accuracy: 0.8288 - val_loss: 0.3853
Epoch 2/10
625/625             4s 6ms/step -
accuracy: 0.8722 - loss: 0.3085 - val_accuracy: 0.8712 - val_loss: 0.3070
Epoch 3/10
625/625             4s 6ms/step -
accuracy: 0.9046 - loss: 0.2422 - val_accuracy: 0.8752 - val_loss: 0.2937
Epoch 4/10
625/625             4s 7ms/step -
accuracy: 0.9252 - loss: 0.1985 - val_accuracy: 0.8724 - val_loss: 0.3123
Epoch 5/10
625/625             4s 6ms/step -
accuracy: 0.9417 - loss: 0.1614 - val_accuracy: 0.8710 - val_loss: 0.3168
Epoch 6/10
625/625             4s 6ms/step -
accuracy: 0.9581 - loss: 0.1272 - val_accuracy: 0.8754 - val_loss: 0.3181
Epoch 7/10
625/625             4s 6ms/step -
accuracy: 0.9711 - loss: 0.0979 - val_accuracy: 0.8728 - val_loss: 0.3307
Epoch 8/10
625/625             4s 6ms/step -
accuracy: 0.9801 - loss: 0.0737 - val_accuracy: 0.8662 - val_loss: 0.3593
Epoch 9/10
625/625             4s 6ms/step -
accuracy: 0.9858 - loss: 0.0540 - val_accuracy: 0.8640 - val_loss: 0.3803
Epoch 10/10
625/625             4s 6ms/step -
accuracy: 0.9919 - loss: 0.0381 - val_accuracy: 0.8672 - val_loss: 0.3895
782/782             2s 2ms/step -
accuracy: 0.8596 - loss: 0.4002
Test accuracy: 0.8596
Embedding (output_dim=16) vs One-hot: 0.8596 vs 0.6922

Model summary for maxlen=200, output_dim=64:

Model: "sequential_22"
```

Layer (type)	Output Shape	Param #
embedding_16 (Embedding)	(None, 200, 64)	640,000
flatten_16 (Flatten)	(None, 12800)	0
dense_22 (Dense)	(None, 1)	12,801

Total params: 652,801 (2.49 MB)

Trainable params: 652,801 (2.49 MB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
625/625             7s 11ms/step -
accuracy: 0.7426 - loss: 0.5092 - val_accuracy: 0.8216 - val_loss: 0.3843
Epoch 2/10
625/625             7s 11ms/step -
accuracy: 0.8860 - loss: 0.2784 - val_accuracy: 0.8714 - val_loss: 0.3104
Epoch 3/10
625/625             8s 13ms/step -
accuracy: 0.9345 - loss: 0.1833 - val_accuracy: 0.8614 - val_loss: 0.3257
Epoch 4/10
625/625             8s 13ms/step -
accuracy: 0.9709 - loss: 0.1041 - val_accuracy: 0.8612 - val_loss: 0.3386
Epoch 5/10
625/625             8s 13ms/step -
accuracy: 0.9891 - loss: 0.0525 - val_accuracy: 0.8602 - val_loss: 0.3686
Epoch 6/10
625/625             7s 12ms/step -
accuracy: 0.9952 - loss: 0.0253 - val_accuracy: 0.8514 - val_loss: 0.4208
Epoch 7/10
625/625             10s 11ms/step -
accuracy: 0.9976 - loss: 0.0130 - val_accuracy: 0.8472 - val_loss: 0.4623
Epoch 8/10
625/625             8s 13ms/step -
accuracy: 0.9990 - loss: 0.0073 - val_accuracy: 0.8452 - val_loss: 0.4934
Epoch 9/10
625/625             9s 12ms/step -
accuracy: 0.9993 - loss: 0.0048 - val_accuracy: 0.8456 - val_loss: 0.5218
Epoch 10/10
625/625             7s 11ms/step -
accuracy: 0.9997 - loss: 0.0031 - val_accuracy: 0.8382 - val_loss: 0.5751
```

```
782/782             2s 3ms/step -
accuracy: 0.8410 - loss: 0.5714
Test accuracy: 0.8410
Embedding (output_dim=64) vs One-hot: 0.8410 vs 0.6922
```